WIPRO RGA PROGRAM 2025 — CAPSTONE PROJECT REPORT

NexCare

Interactive Dashboard System for Customer Service Operations

Submitted by: **Rishitha Manyam**

Program: Java Full Stack Development

Submission Date: June 2026

Technology Stack: Angular 17, Spring Boot 3.2.5, H2 Database

Table of Contents

01 Abstract

NexCare is a full-stack, role-based web application designed to digitise and centralise customer service operations for telecom companies operating in India. The system replaces fragmented, manual processes — spreadsheets, phone-based escalation, and email communication — with a single secure platform that serves four distinct user roles: Administrator, Manager, Service Representative, and Customer. Built using Angular 17 on the frontend and Spring Boot 3.2.5 on the backend, NexCare implements JSON Web Token (JWT) based stateless authentication, role-based authorization enforced at both the UI and API layers, real-time ticket lifecycle tracking with automatic performance metric calculation, and a self-service chatbot for first-line customer support. The application is fully localised for the Indian market, supporting Indian cities, INR currency conventions, and Indian phone number formats. This report documents the problem motivating the project, the system design, the technologies used, the implementation approach, the security model, and the testing carried out to validate the application before deployment.

02 Introduction

Customer service operations in the telecom sector involve a continuous cycle of complaint registration, assignment, resolution, and feedback. As the customer base grows, manual coordination between customers, service representatives, and management becomes a bottleneck — leading to delayed responses, lost visibility into outstanding issues, and an inability to measure representative performance objectively.

NexCare was conceived as a capstone project to address this gap by building a complete, production-style web application that any telecom service provider could realistically deploy. The project was undertaken individually as part of the Wipro RGA Program 2025, applying the Java Full Stack curriculum — specifically Spring Boot for backend services and Angular for the client-facing application — to a real-world operational problem.

Problem Statement

Telecom customer service teams in India face the following recurring issues:

- No unified system — tickets are tracked across spreadsheets, email threads, and verbal handovers, resulting in lost or duplicated work.
- Managers have no real-time visibility into team workload or individual representative performance (response time, resolution time, customer satisfaction).
- Customers have no way to track the status of their own complaints, leading to repeated follow-up calls that consume representative time.
- There is no role-based data isolation — any staff member can potentially view data belonging to any customer, creating both a privacy and an operational risk.
- Service outages are communicated informally, leaving customers in affected areas uninformed while still being able to raise duplicate tickets for a known issue.

04 Objectives of the Project

1. Design and implement a single web application serving four distinct user roles with strict role-based access control.
2. Provide customers with self-service capability to raise, track, and rate support tickets.
3. Provide representatives with a focused workspace to manage assigned tickets and search customer history.
4. Provide managers with team-level analytics — ticket volume, response time, and resolution time per representative.
5. Provide administrators with full operational control — employee management, ticket oversight, and outage reporting.
6. Secure all API access using stateless JWT authentication so the system can scale horizontally without server-side session storage.
7. Automatically calculate and surface key service-level metrics (response time, resolution time, customer satisfaction rating) without manual data entry.
8. Localise the entire application for the Indian market.

05 Existing System and Limitations

In most small-to-mid-sized telecom service desks observed informally, the "system" in use is a combination of:

Tool used today	Limitation
Excel / Google Sheets for ticket tracking	No real-time updates, no access control, prone to accidental overwrites, no audit trail
Phone calls / WhatsApp for status updates	Not searchable, not measurable, consumes representative time repeatedly
Email for outage announcements	Not targeted — every customer receives the same email regardless of whether they are affected
Manual performance reviews	Subjective, infrequent, not backed by objective response/resolution time data

These limitations directly motivated the design decisions in NexCare: every weakness above maps to a specific feature in the proposed system (described in Section 6).

06 Proposed System

NexCare proposes a single web application with four role-specific dashboards, backed by one shared database and one secured REST API. Key design decisions:

- **Single Page Application (SPA):** Angular handles all UI rendering client-side; the Spring Boot backend serves only JSON data, never HTML — keeping the two layers cleanly separated and independently scalable.
- **Stateless authentication:** JWT tokens carry all the information needed to authorize a request, removing the need for server-side session storage and making the backend horizontally scalable.
- **Role-based dashboards:** Rather than one generic dashboard with conditional rendering, each role gets a dedicated component tree — simplifying the UI logic and reducing the risk of accidentally exposing UI elements to the wrong role.

- **Automatic metric calculation:** Response time and resolution time are computed server-side at the moment a ticket's status changes, removing any possibility of manual data entry error.
- **Targeted outage alerts:** Outages are tagged with a city; only customers located in that city see the alert banner, reducing noise for unaffected customers.

07 System Requirements

Hardware Requirements (Development Machine)

Component	Minimum
RAM	4 GB (8 GB recommended)
Disk Space	2 GB free
Processor	Dual-core, 2 GHz or higher

Software Requirements

Software	Version	Purpose
Java Development Kit (JDK)	17 or higher	Compiles and runs the Spring Boot backend
Node.js	18.x LTS or higher	Runs the Angular build tooling and dev server
Apache Maven	3.9.x (or bundled wrapper)	Builds the backend, manages Java dependencies
Angular CLI	17.x	Builds and serves the frontend
H2 Database	Embedded (file mode)	Persistent storage, no separate installation needed
Visual Studio Code / Eclipse	Latest	IDE used for development
Git	Latest	Version control and source hosting on GitHub

08 System Design and Architecture

NexCare follows a three-tier architecture, cleanly separating presentation, business logic, and data persistence:

Tier	Technology	Responsibility
Presentation	Angular 17 (Standalone Components)	Renders all UI, manages client-side routing, calls backend REST APIs
Application / Service	Spring Boot 3.2.5 REST Controllers and Services	Validates requests, enforces business rules, orchestrates data access
Data	Spring Data JPA + H2 File Database	Persists Users, Tickets, Outages, and Notifications

Communication between the frontend and backend happens exclusively over HTTP using JSON payloads. During development, Angular's dev server (port 4200) proxies all `/api` requests to the Spring Boot server (port 8080) via `proxy.conf.json`, avoiding cross-origin resource sharing (CORS) issues without weakening the production CORS policy.

Architectural Patterns Used

- **MVC (Model-View-Controller)** on the backend — Controllers handle HTTP, Services hold business logic, Repositories handle persistence, Models represent data.
- **Repository Pattern** — Spring Data JPA repositories abstract all SQL behind Java interfaces.
- **DTO (Data Transfer Object) Pattern** — Request and response bodies use dedicated DTO classes rather than exposing JPA entities directly over the API.
- **Functional Interceptor / Guard Pattern (Angular)** — Cross-cutting concerns (attaching the JWT token, checking authentication) are implemented as standalone functions rather than classes, following Angular 17's modern style.
- **Observer Pattern (RxJS)** — Used for cross-component communication, e.g. the sidebar signalling the chatbot to open via a shared `ChatService`.
- **Circuit Breaker Pattern (Resilience4j)** — Applied to the ticket retrieval service so a failure does not cascade into a complete application crash.

09 Database Design

The application uses an H2 file-based relational database. Spring Data JPA's `ddl-auto=update` setting automatically generates and updates the schema from the Java entity classes, removing the need to write or maintain SQL DDL scripts manually.

Entity Relationship Summary

Entity	Key Relationships
User	A Representative references their Manager via `managerId`. A Customer is referenced by Tickets via `customerId`.
Ticket	References the raising Customer (`customerId`), the assigned Representative (`repId`), and that Representative's Manager (`managerId`) — denormalised for fast manager-level reporting.
Outage	Independent entity, filtered by `location` when customers query for active outages in their city.
Notification	References the recipient via `userId`; created automatically by the Ticket and Outage services as side effects of state changes.

USERS Table

Column	Type	Notes
id	BIGINT	Primary key, auto-increment
name, email, password	VARCHAR	email is unique; password stored as a BCrypt hash, never plaintext
role	VARCHAR (enum)	ADMIN / MANAGER / REPRESENTATIVE / CUSTOMER
phone, location	VARCHAR	Indian phone format; Indian city name
manager_id	BIGINT	Self-referencing — links a representative to their manager
first_login	BOOLEAN	Forces a password change on a representative's first sign-in

TICKETS Table

Column	Type	Notes
id, customer_id, rep_id, manager_id	BIGINT	Foreign references (logical, not enforced FK constraints)
domain, subject, description	VARCHAR	Ticket classification and content
status	VARCHAR (enum)	OPEN / IN_PROGRESS / CLOSED
created_at, first_response_at, resolved_at	TIMESTAMP	Timeline used to compute SLA metrics
response_time_hrs, resolution_time_hrs	DOUBLE	Computed automatically — never entered manually
rating	INTEGER	1–5, set by the customer only after the ticket is CLOSED

10 Implementation Details

10.1 Backend Implementation

The backend is organised into five layers under the package `com.wipro.csd`:

- **model** — JPA entity classes (User, Ticket, Outage, Notification) mapped directly to database tables.
- **repository** — Spring Data JPA interfaces providing CRUD operations and custom derived queries (e.g. `findByCustomerIdOrderByCreatedAtDesc`) without any hand-written SQL.
- **service** — Business logic. For example, `TicketService.updateStatus()` calculates response and resolution times by comparing timestamps whenever a representative changes a ticket's status.
- **controller** — REST endpoints exposed under `/api/**`, annotated with `@PreAuthorize` to enforce role checks at the method level in addition to the global security filter.
- **config / filter** — Cross-cutting infrastructure: `SecurityConfig` (authorization rules), `JwtUtil` (token generation/validation), `JwtAuthFilter` (per-request token check), and `DataInitializer` (seeds demo data on first run via `@PostConstruct`).

All model and DTO classes were implemented with explicit getters and setters rather than relying on a code-generation library, ensuring the project compiles identically regardless of the Java Development Kit version installed on the build machine.

10.2 Frontend Implementation

The frontend is built using Angular 17's standalone component model — there is no `NgModule` anywhere in the application. Each route is lazy-loaded via `loadComponent()`, meaning the JavaScript for, say, the Admin dashboard is only downloaded by the browser when an admin actually navigates there, keeping the initial page load fast.

Key implementation choices:

- **Hash-based routing** (`withHashLocation()`) was chosen so the application can be hosted on static file servers (such as GitHub Pages) without server-side URL rewriting.
- **A functional HTTP interceptor** automatically attaches the JWT token to every outgoing request, so no individual service method needs to handle authentication headers manually.
- **A functional route guard** checks both the presence of a token and the user's role before allowing navigation to a protected route, redirecting unauthenticated or unauthorized users back to the login

page.

- **Tab navigation within a dashboard** (e.g. Admin's Dashboard / Employees / Tickets / Outages tabs) is implemented using Angular Router query parameters rather than component-internal state, making every tab bookmarkable and refresh-safe.
- **Cross-component communication** (the sidebar opening the chatbot) uses a shared injectable service with an RxJS `Subject`, which is the idiomatic Angular pattern for sibling components that do not have a direct parent-child relationship.

10.3 Code Snapshot — Automatic SLA Metric Calculation

```
if (newStatus == TicketStatus.IN_PROGRESS && ticket.getFirstResponseAt() == null) {
    LocalDateTime now = LocalDateTime.now();
    ticket.setFirstResponseAt(now);
    double hrs = ChronoUnit.MINUTES.between(ticket.getCreatedAt(), now) / 60.0;
    ticket.setResponseTimeHrs(Math.round(hrs * 10.0) / 10.0);
}
```

This snippet, taken from `TicketService.updateStatus()`, demonstrates how the response time metric is derived purely from timestamps already present on the entity — no separate manual entry step exists in the system, which removes a whole category of potential data-quality issues found in the spreadsheet-based approach described in Section 5.

11 Security Implementation

Security in NexCare operates at three layers, each independent of the others so that a failure in one layer does not compromise the system:

1. **Transport-level:** All credentials are submitted over HTTP POST bodies, never in URLs, preventing accidental leakage via browser history or server access logs.
2. **Authentication:** Passwords are hashed using BCrypt before storage; the plaintext password is never persisted. On successful login, a JWT signed with HMAC-SHA384 is issued, embedding the user's id, role, name, and a 24-hour expiry.
3. **Authorization:** Every protected request passes through `JwtAuthFilter`, which validates the token's signature and expiry before populating Spring Security's context. Method-level `@PreAuthorize` annotations then enforce that, for instance, only a user with role `CUSTOMER` can call the "create ticket" endpoint.

On the frontend, a route guard provides a first line of defence by preventing the UI from even attempting to render a page the logged-in user is not authorized for — this is a usability feature, not a security boundary, since the authoritative check always happens on the server.

12 Testing

The application was tested manually across the following scenarios prior to demonstration:

Test Case	Expected Result	Outcome
Login with valid Admin credentials	JWT issued, redirected to Admin dashboard	Pass
Login with incorrect password	401 Unauthorized, error message shown	Pass
Customer attempts to access /admin directly via URL	Redirected to login by route guard	Pass
Customer raises a new ticket	Ticket created with status OPEN; all assigned representatives notified	Pass

Representative updates ticket to IN_PROGRESS	responseTimeHrs calculated automatically; customer notified	Pass
Representative updates ticket to CLOSED	resolutionTimeHrs calculated; customer can now rate the ticket	Pass
Admin reports an outage in Mumbai	Only customers located in Mumbai see the alert banner	Pass
Token expiry after 24 hours	Subsequent API calls return 401; user redirected to login	Pass
Search ticket by customer phone number	Matching tickets returned to representative within 400ms debounce	Pass
Concurrent restart of the application	DataInitializer does not duplicate seed data on second run	Pass

13 Results and Outcome

The completed application successfully demonstrates all objectives defined in Section 4. Each of the four roles has a functioning, secured dashboard. Tickets move through their full lifecycle with automatically computed performance metrics. Outage alerts are correctly scoped to the affected city only. The chatbot provides immediate responses to common queries without requiring a ticket to be raised for routine questions.

From a non-functional perspective, the application starts from a clean checkout with two commands and seeds its own demonstration data, meaning a new evaluator can have a fully working, populated system running within roughly three minutes — addressing a real practical concern for a project that needs to be demonstrated live.

14 Challenges Faced and Solutions

Challenge	Solution
Sibling Angular components (sidebar and chatbot) needed to communicate without a parent-child relationship	Introduced a shared injectable `ChatService` using an RxJS `Subject` as an event bus
Tab navigation using `scrollIntoView` was unreliable across page refreshes	Replaced with Angular Router query parameters, making every tab state bookmarkable and refresh-safe
Chart.js canvas elements were not yet rendered when charts were initialised inside an `@if` block	Deferred chart creation with a short `setTimeout` after the tab becomes active, ensuring the DOM element exists first
Cross-origin requests blocked between Angular (port 4200) and Spring Boot (port 8080) during development	Configured an Angular dev-server proxy (`proxy.conf.json`) to forward `/api` requests transparently
A code-generation library used for entity boilerplate behaved inconsistently across different JDK versions on different development machines	Replaced all generated code with explicit, hand-written getters and setters, eliminating the dependency entirely and guaranteeing identical builds on any machine

15 Conclusion

NexCare demonstrates the practical application of the Java Full Stack curriculum to a realistic business problem. The project required integrating a secure authentication scheme, a role-based authorization model, a relational data design, and a modern reactive frontend into a single coherent system — mirroring the kind of work expected in an industry setting rather than an isolated academic exercise. The resulting application is secure, internally consistent, and fully documented, providing both a working demonstration and a body of supporting material (setup guides, a technical Q&A reference, and this report) suitable for evaluation.

16 Future Scope

- Migrate from the embedded H2 file database to a production-grade RDBMS such as PostgreSQL or MySQL for true concurrent multi-user deployment.
- Add email or SMS notifications for ticket status changes, in addition to the in-app notification bell.
- Introduce file attachments on tickets (e.g. photos of damaged equipment).
- Add a refresh-token mechanism to allow session renewal without forcing a full re-login after 24 hours.
- Extend the chatbot with a more sophisticated NLP-based intent classifier in place of the current keyword-matching approach.
- Add automated unit and integration tests (JUnit for the backend, Karma/Jasmine for the frontend) to support continuous integration.

17 References

- Spring Boot Reference Documentation — https://docs.spring.io/spring-boot/
- Spring Security Reference Documentation — https://docs.spring.io/spring-security/
- Angular Official Documentation — https://angular.dev
- JJWT (Java JWT) Library Documentation — https://github.com/jwtk/jjwt
- Resilience4j Documentation — https://resilience4j.readme.io
- H2 Database Engine Documentation — https://www.h2database.com
- Chart.js Documentation — https://www.chartjs.org

Rishitha Manyam

Project Author

Date: June 2026

Wipro RGA Program 2025